26

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Ret Politikaları (Rejection Policies): Abort, CallerRuns, Discard & DiscardOldest

Task Rejection Policies: AbortPolicy, CallerRuns, Discard & DiscardOldest

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              THREAD POOL 4 RET POLİTİKASI (REJECTION POLICIES)              │
└──────────────────────────────────────────────────────────┘

KUYRUK DOLU VE MAX THREAD'LER MEŞGUL İKEN YENİ GÖREV GELDİĞİNDE:

1. AbortPolicy (Varsayılan / Sert):
   ──► RejectedExecutionException fırlatır! "Ben tıkandım, işlemi reddediyorum!"

2. CallerRunsPolicy (En Popüler / Doğal Backpressure):
   ──► Görevi havuz değil, görevi havuza GÖNDEREN THREAD (Örn: Tomcat HTTP) çalıştırır!
   ──► Gönderen thread meşgul olunca yeni istek kabul edemez; sistem doğal olarak yavaşlar!

3. DiscardPolicy (Sessiz Çöp):
   ──► Görevi hiçbir hata fırlatmadan SESSİZCE ÇÖPE ATAR! (Örn: Önemsiz log/metrik)

4. DiscardOldestPolicy (En Eskiye Tekme):
   ──► Kuyruğun başındaki en eski görevi silip atar, yeni görevi kuyruğa sokar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir pizzacı dükkanı düşünün. Fırınlar dolu, masalar dolu, kuryeler meşgul. Kapıdan yeni bir müşteri girip pizza istediğinde pizzacı ne yapar? 1) Abort: Müşterinin yüzüne kapıyı çarpar: 'Sipariş alamam!' der (Hata fırlatır). 2) CallerRuns: Müşterinin eline önlük tutuşturur: 'Çok istiyorsan mutfağa gir kendin pişir!' der (İsteği getiren kişi kendi işini kendi yapar, böylece kapıda yeni müşteri birikmez). 3) Discard: Sipariş fişini alır ve müşteri arkasını döndüğü anda çöpe atar (Sessizce yutar). 4) DiscardOldest: Bekleyen en eski müşterinin siparişini iptal eder, yeni gelen müşteriyi sıraya alır!

🇬🇧 ENGLISH (ELI5)

Imagine a bakery operating at maximum capacity. The oven is full, counters overflowing. When a customer orders a cake: 1) AbortPolicy: The baker slams the shutter and shouts 'Denied!' (`RejectedExecutionException`). 2) CallerRunsPolicy: The baker hands the customer an apron: 'Bake it yourself!' The caller executes the task on their own thread, naturally throttling new orders. 3) DiscardPolicy: The baker silently drops the order slip into the trash (silent drop). 4) DiscardOldestPolicy: The baker evicts the oldest stale ticket waiting in line and enqueues the fresh order.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CallerRunsPolicy ile Doğal Backpressure (Geri Basınç): 1) Neden Üretimde CallerRuns Tercih Edilir?: Web sunucularında (Tomcat, Netty, Spring Boot) arka plandaki thread havuzu tıkandığında `CallerRunsPolicy` seçilirse, görevi havuza atan HTTP Acceptor thread'i görevi kendi üzerinde koşturmaya başlar. Bu süre zarfında soketten yeni HTTP istekleri kabul edilemez; işletim sistemi TCP SYN kuyruğunda bekletir veya yük dengeleyiciye (Load Balancer) 'Ben doluyum' sinyali gider. Bu, sistemin OOM çöküşünü önleyen en zarif mimari savunmadır. 2) DiscardOldest Tehlikesi: `PriorityBlockingQueue` ile kullanılırsa, en eski görev yerine en yüksek öncelikli görev çöpe atılabilir!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CallerRunsPolicy & The Elegance of Organic Backpressure: 1) Mitigating Cascading Failures: When a downstream worker pool saturates, `CallerRunsPolicy` delegates execution to the invoking thread (e.g. Tomcat/Jetty worker). Because the calling thread is now occupied executing computation, it stops accepting new requests from the network socket. The TCP receive buffer fills, and upstream load balancers detect backpressure, rerouting traffic away from the saturated node. 2) DiscardOldest Hazard: When paired with a `PriorityBlockingQueue`, `DiscardOldest` evicts the head of the queue, which is the HIGHEST priority task, not the oldest chronologically!

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: CallerRunsPolicy ile Doğal Backpressure Gösterimi
import java.util.concurrent.*;

public class RejectionPolicyDemo {
    void main() {
        // 1 Thread, 1 Kuyruk Kapasitesi (Toplam 2 görev alabilir)
        var executor = new ThreadPoolExecutor(
            1, 1, 0L, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(1),
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure kalkanı
        );

        for (int i = 1; i <= 4; i++) {
            final int taskId = i;
            executor.execute(() -> {
                IO.println("Görev #" + taskId + " çalıştırılıyor. Thread: " +
Thread.currentThread().getName());
                try { Thread.sleep(200); } catch (InterruptedException ignored) {}
            });
        }

        executor.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'CallerRunsPolicy kullandığımızda ana thread kilitlenebilir mi?' diye sorar. Yanıt: 'Evet! Eğer havuza atılan görev uzun süren bir işlemse veya bloke oluyorsa, ana thread (ör. UI thread veya Netty event-loop) o görevi çalıştırırken donar ve tüm uygulamanın tepki süresi (responsiveness) durur!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Architectural Hazard: 'Can CallerRunsPolicy degrade the application main loop?' Answer: 'YES! If the calling thread is a Netty EventLoop or a UI dispatcher, forcing it to run long-running CPU or blocking I/O tasks halts the entire event-loop, starving all other multiplexed channels.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• AbortPolicy varsayılandır ve RejectedExecutionException fırlatır.

• CallerRunsPolicy, çağıran thread'i çalıştırarak doğal bir backpressure kalkanı oluşturur.

• DiscardPolicy sessizce görevi yutar; yalnızca önemsiz telemetri loglarında kullanılmalıdır.

🎯 Key Takeaways for the Interview

• AbortPolicy is the default, throwing RejectedExecutionException on saturation.

• CallerRunsPolicy creates natural backpressure by forcing the submitting thread to execute the work.

• DiscardPolicy silently drops tasks and should only be applied to dispensable metrics.